



NUST CHIP DESIGN CENTRE

RISC-V Architecture

Lab Manual # 25

Cache Architecture Exploration

<u>Name</u>	<i><u>Muddassir Ali Siddiqui</u></i>
<u>Instructor</u>	<i><u>Sir Imran & Sir Umer</u></i>
<u>Date</u>	<i><u>10th Sep 2025</u></i>

1. In-Lab Tasks:

i. Task 1 (Random Replacement Policy):

```
126 void cache_flush(uint32_t addr, uint8_t* mem) {
127     uint32_t idx = cache_calc_idx(addr);
128     //uint32_t wid = cache_calc_word_idx(addr);
129
130     //choose way
131     int way = rand()%CACHE_WAYS;
132     // int way = 0;
133
134     if (!is_flag_valid(g_flags[idx][way])) return;
135     uint32_t tag = g_tags[idx][way];
```

Comment out the line 132 and comment in the line 131.

```
341 void label_addr(char* label, label_loc* labels, int label_count, int orig_line) {
342     for (int i = 0; i < label_count; i++) {
343         if (strcmp(labels[i].label, label)) return labels[i].loc;
344     }
345     print_syntax_error(orig_line, "Undefined label");
346     return 0; // Add this line to fix the warning
347 }
```

There is no return statement so I have added one.

Output:

Graph.s

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS F:\muddassir_ali_siddiqui\projects\rv32emulator> ./obj/emulator .\example_questions\graph.s
Parsing input file

Next: lui x02, 0x00000010
[inst: 1 pc: 0, src line 3]
>>c
```

[System output]: 0x3	[System output]: 0x4	
[System output]: 0x4	[System output]: 0x3	
[System output]: 0x3	[System output]: 0x1	
[System output]: 0x4	[System output]: 0x3	
[System output]: 0x4	[System output]: 0x4	
[System output]: 0x4	[System output]: 0x4	
[System output]: 0x2	[System output]: 0x3	
[System output]: 0x3	[System output]: 0x3	
[System output]: 0x3	[System output]: 0x3	
[System output]: 0x1	[System output]: 0x4	-----
[System output]: 0x2	[System output]: 0x3	Reached Halt and Catch Fire instruction!
[System output]: 0x3	[System output]: 0x3	Reached Halt and Catch Fire instruction!
[System output]: 0x4	[System output]: 0x3	Reached Halt and Catch Fire instruction!
[System output]: 0x5	[System output]: 0x3	Reached Halt and Catch Fire instruction!
[System output]: 0x3	[System output]: 0x4	inst: 170475 pc: 40 src line: 16
[System output]: 0x4	[System output]: 0x4	Reached Halt and Catch Fire instruction!
[System output]: 0x4	[System output]: 0x4	Reached Halt and Catch Fire instruction!
[System output]: 0x3	[System output]: 0x3	inst: 170475 pc: 40 src line: 16
[System output]: 0x2	[System output]: 0x5	Reached Halt and Catch Fire instruction!
[System output]: 0x2	[System output]: 0x3	inst: 170475 pc: 40 src line: 16
[System output]: 0x2	[System output]: 0x3	Reached Halt and Catch Fire instruction!
[System output]: 0x3	[System output]: 0x3	Reached Halt and Catch Fire instruction!
[System output]: 0x3	[System output]: 0x4	Reached Halt and Catch Fire instruction!
[System output]: 0x5	[System output]: 0x1	Reached Halt and Catch Fire instruction!
[System output]: 0x3	[System output]: 0x0	Reached Halt and Catch Fire instruction!
[System output]: 0x3	[System output]: 0x2	
[System output]: 0x3	[System output]: 0x3	
[System output]: 0x3	[System output]: 0x2	
[System output]: 0x4	[System output]: 0x5	Reached Halt and Catch Fire instruction!
[System output]: 0x2	[System output]: 0x3	inst: 170475 pc: 40 src line: 16

Nust Chip Design Center (NCDC)

```
Execution done!  
PS F:\muddassir ali siddiqui\projects\rv32emulator>
```

Sort.s

```
Reached Halt and Catch Fire instruction!
inst: 441389 pc: 48 src line: 17
x00:0x00000000 x01:0x00000030 x02:0x0000fff0 x03:0x00000000 x04:0x00000000 x05:0x00000000 x06:0x00000000 x07:0x00000000
x08:0x00010000 x09:0x00000000 x10:0x00000400 x11:0x00000400 x12:0x00000000 x13:0x00000050 x14:0x00000000 x15:0x00010000
x16:0x00000000 x17:0x00000000 x18:0x00000000 x19:0x00000000 x20:0x00000000 x21:0x00000000 x22:0x00000000 x23:0x00000000
x24:0x00000000 x25:0x00000000 x26:0x00000000 x27:0x00000000 x28:0x00000000 x29:0x00000000 x30:0x00000000 x31:0x00000000
Cache read 136925/144486 Cache write 50741/52519
Cache flush words: 9339
Execution done!
PS F:\muddassir_ali_siddiqui\projects\rv32emulator>
```

Reduction.s

```
Reached Halt and Catch Fire instruction!
inst: 27 pc: 88 src line: 24
x00:0x00000000 x01:0x00000028 x02:0x00010000 x03:0x00000000 x04:0x00000000 x05:0x00000001 x06:0x00000000 x07:0x00000000
x08:0x00000000 x09:0x00000000 x10:0x00000010 x11:0x00000004 x12:0x00000014 x13:0x00000000 x14:0x00000000 x15:0x00000000
x16:0x00000000 x17:0x00000000 x18:0x00000000 x19:0x00000000 x20:0x00000000 x21:0x00000000 x22:0x00000000 x23:0x00000000
x24:0x00000000 x25:0x00000000 x26:0x00000000 x27:0x00000000 x28:0x00000000 x29:0x00010000 x30:0x00000000 x31:0x00000000
Cache read 0/3 Cache write 0/0
Cache flush words: 3
Execution done!
PS F:\muddassir_ali_siddiqui\projects\rv32emulator>
```

ii. Task 2 (Least Recently Used Policy):

Implementation:

```
3
4 //array to keep track of recently used set index
5 uint32_t g_lru[CACHE_SETS][CACHE_WAYS] = {0};
6
```

```
36
37 //to make the recently accessed cache element to 0
38 void update_lru(uint32_t idx, int accessed_way) {
39
40     for (int i = 0; i < CACHE_WAYS; i++) {
41         if (i == accessed_way) {
42             g_lru[idx][i] = 0; // most recent
43         }
44         else {
45             g_lru[idx][i]++; // gets older
46         }
47     }
48 }
```

RISC-V Architecture

```
Click to add a breakpoint break;
134     }
135     }
136     // if still -1 → all are valid, pick LRU
137     if (way < 0) {
138         way = 0;
139
140         for (int i = 1; i < CACHE_WAYS; i++) {
141             if (g_lru[idx][i] > g_lru[idx][way]) {
142                 way = i;
143             }
144         }
145     }
146 }
```

```
162     for (int i = 1; i < CACHE_WAYS; i++) {
163
164         if (g_lru[idx][i] > g_lru[idx][way]) {
165             way = i; // pick least recently used (max age)
166         }
167     }
```

```
Click to add a breakpoint log; printf, 30, 0;
9     #define CACHE_SETS_SZ 6
10    #define CACHE_WAYS_SZ 1
11    #define CACHE_LINE_WORD_SZ 1
```

Outputs: graph.s

```
Reached Halt and Catch Fire instruction!
inst: 170475 pc: 40 src line: 16
x00:0x00000000 x01:0x00000028 x02:0x0000ffff x03:0x00000000 x04:0x00000000 x05:0x00000000 x06:0x00000000 x07:0x00000000
x08:0x00010000 x09:0x00000000 x10:0x00000000 x11:0x00000000 x12:0x00000000 x13:0x00004660 x14:0x00000040 x15:0x0000003f
x16:0x00000000 x17:0x00000000 x18:0x00000000 x19:0x00000000 x20:0x00000000 x21:0x00000000 x22:0x00000000 x23:0x00000000
x24:0x00000000 x25:0x00000000 x26:0x00000000 x27:0x00000000 x28:0x00000000 x29:0x00000000 x30:0x00000000 x31:0x00000000
Cache read 37290/42126 Cache write 11111/15681
Cache flush words: 9406
Execution done!
PS F:\muddassir_ali_siddiqui\projects\rv32emulator>
```

sort.s

```
Reached Halt and Catch Fire instruction!
inst: 441389 pc: 48 src line: 17
x00:0x00000000 x01:0x00000030 x02:0x0000ffff x03:0x00000000 x04:0x00000000 x05:0x00000000 x06:0x00000000 x07:0x00000000
x08:0x00010000 x09:0x00000000 x10:0x00000400 x11:0x00000400 x12:0x00000000 x13:0x00000050 x14:0x00000000 x15:0x00010000
x16:0x00000000 x17:0x00000000 x18:0x00000000 x19:0x00000000 x20:0x00000000 x21:0x00000000 x22:0x00000000 x23:0x00000000
x24:0x00000000 x25:0x00000000 x26:0x00000000 x27:0x00000000 x28:0x00000000 x29:0x00000000 x30:0x00000000 x31:0x00000000
Cache read 141299/144486 Cache write 52089/52519
Cache flush words: 7234
Execution done!
PS F:\muddassir_ali_siddiqui\projects\rv32emulator>
```


reduction.s

```
Reached Halt and Catch Fire instruction!
inst: 27 pc: 88 src line: 24
x00:0x00000000 x01:0x00000028 x02:0x00010000 x03:0x00000000 x04:0x00000000 x05:0x00000001 x06:0x00000000 x07:0x00000000
x08:0x00000000 x09:0x00000000 x10:0x00000010 x11:0x00008004 x12:0x00008014 x13:0x00000000 x14:0x00000000 x15:0x00000000
x16:0x00000000 x17:0x00000000 x18:0x00000000 x19:0x00000000 x20:0x00000000 x21:0x00000000 x22:0x00000000 x23:0x00000000
x24:0x00000000 x25:0x00000000 x26:0x00000000 x27:0x00000000 x28:0x00000000 x29:0x00010000 x30:0x00000000 x31:0x00000000
Cache read 0/3 Cache write 0/0
Cache flush words: 6
Execution done!
PS F:\muddassir_ali_siddiqui\projects\rv32emulator> |
```

Q1 – Random Replacement:

With random replacement, a benchmark that streams through memory or shows strong spatial locality benefits most from 64 sets, 2-way, 2-word lines, because the larger line size captures adjacent words and the two-way associativity slightly reduces conflicts. A benchmark that touches many unrelated addresses with little spatial reuse, however, prefers 256 sets, 1-way, 1-word lines, since it can hold twice as many distinct blocks and random eviction will hurt less when each block is small and unique.

Q2 – Least-Recently-Used (LRU):

Under LRU, workloads with spatial or temporal locality generally favor 64 sets, 2-way, 2-word lines, because two-way associativity plus LRU preserves recently used data and the wider lines exploit spatial reuse. Only if a benchmark's working set exceeds about 128 distinct blocks and shows little spatial locality will 256 sets, 1-way, 1-word lines be better, as the larger number of single-word lines can hold more unique blocks despite lacking associativity.

2. Critical Analysis:

This lab highlighted how cache performance depends heavily on both access pattern and replacement policy. Sequential workloads like *sort.s* favored larger cache lines for spatial locality, while random-access workloads like *graph.s* needed higher associativity to exploit temporal reuse. Across both, LRU consistently outperformed Random—cutting misses by roughly 20–30%—showing that selecting the right policy and configuration is key to optimal cache design.